DETAILED PROJECT REPORT

NexTask

Task management web application

HTML · CSS · JavaScript · Python Flask · SQLite

PROJECT FOCUS

NexTask centralizes task organization, API communication, persistent storage, Trash management, and JSON backup workflows in one local web application.

01 **Organize work**

Create, classify, update, complete, and review tasks.

02 **Protect records**

Use SQLite persistence, recoverable Trash, and JSON backup operations.

03 **Connect the stack**

Demonstrate frontend, backend, API, database, and version-control integration.

Made with GAMMA

01 · FOUNDATIONS

Introduction, problem, and objectives

Daily work becomes difficult to monitor when activities, deadlines, priorities, and completion states are distributed across notes or informal lists. NexTask provides one place to create, organize, update, complete, delete, restore, and back up tasks.

OBJECTIVES

01 Provide a simple interface for creating and viewing tasks.

02 Organize tasks using categories, priorities, and due dates.

03 Allow editing and completion-status updates.

04 Provide recoverable Trash management and permanent deletion.

05 Persist records in SQLite and support JSON backup import/export.

06 Demonstrate frontend, backend, API, database, and version-control integration.

Made with GAMMA

02 · BOUNDARIES AND TOOLS

Scope, technology stack, and requirements

The application covers task creation, listing, editing, completion, soft deletion, restoration, permanent deletion, settings, and JSON backup operations.

Intended for personal use and local development. Accounts, multi-user synchronization, and hosted production deployment are outside the current scope.

TECHNOLOGY RESPONSIBILITIES

Layer	Responsibility
HTML/CSS/JavaScript	Page structure, styling, interactions, and API requests.
Python, Flask, Flask-CORS	Backend routes, request processing, and local cross-origin communication.
SQLite	Relational persistence for task records.
Git and GitHub	Version control and repository hosting.

REQUIREMENTS

A compatible computer, at least 4 GB RAM suggested, Python 3, a modern browser, a code editor, and Flask dependencies installed from `requirements.txt` are sufficient for local development.

Made with GAMMA

03 · STRUCTURE

System architecture and request flow

NexTask follows a client-server architecture. The browser frontend communicates with a Flask backend through HTTP and JSON API requests. The backend processes requests, calls model functions, and reads or writes the SQLite database.

01

Browser

Presents HTML, CSS, and JavaScript interfaces.

02

API request

JavaScript sends a request to a Flask API endpoint.

03

Route handler

Validates and processes the request.

04

Model

Performs the required database operation.

05

SQLite

Stores or retrieves the task record.

06

Response

Returns JSON for the interface to display.

LOGICAL ARCHITECTURE

Browser Frontend → HTTP/JSON API → Flask Backend → Model Operations → SQLite Database

Separating interface, routes, models, and storage makes the project easier to understand, test, and extend.

Made with GAMMA

4. Modules and user workflows

Dashboard / Task Interface

Loads and displays task information returned by the backend.

Task Management

Creates tasks, retrieves lists and details, edits fields, and changes completion status.

All Tasks

Provides a view of saved tasks and their management actions.

Trash

Displays soft-deleted tasks and supports restore, permanent deletion, and bulk clearing.

Settings

Manages theme and default priority or category preferences.

Backup

Exports task data and imports JSON backup data.

TYPICAL WORKFLOW

1. Create a task with a title, description, category, priority, and due date.
2. Review or edit the task from the task list.
3. Mark it completed when the work is finished.
4. Move it to Trash when it is no longer active.
5. Restore it when needed, or permanently delete it.

Made with GAMMA

Database design and data management

The main SQLite table is named **tasks**. It stores task content, classification, dates, status flags, and creation time.

Field	Definition and purpose
id	INTEGER PRIMARY KEY AUTOINCREMENT; unique identifier.
title	TEXT NOT NULL; required task title.
description	TEXT; optional description, default empty string.
category	TEXT; default Other.
priority	TEXT; default Medium.
due_date	TEXT; optional due date.
completed	INTEGER; completion flag using 0 or 1.
deleted	INTEGER; soft-deletion flag using 0 or 1.
created_at	TEXT NOT NULL; creation timestamp.

The **completed** and **deleted** flags distinguish active, completed, and deleted records without immediately removing deleted data.

Made with GAMMA

Implementation and local execution

Frontend implementation

HTML pages define structure, CSS controls presentation, and JavaScript performs API calls and updates the interface. Separate scripts support task pages, Trash, and settings.

Backend and database

The Flask application initializes the database, configures CORS, registers task routes, and exposes a health endpoint. A database helper creates connections, while `CREATE TABLE IF NOT EXISTS` allows schema initialization when the application starts.

Backup implementation

The export feature produces a JSON backup file. The import endpoint receives and processes backup data so tasks can be restored or imported.

RUN COMMANDS

1. From the backend directory, run `python app.py`.
2. From the frontend directory, run `python -m http.server 8000`.
3. Open `http://127.0.0.1:8000` for the frontend.
4. The Flask backend runs at `http://127.0.0.1:5000`.

Made with GAMMA

07 · VERIFICATION

Testing and verification

Testing verifies the main task-management workflows and frontend-backend integration. The supplied test summary records each scenario as passed as tested.

01 **TC-01:** Create a valid task; it appears in the task list.

02 **TC-02:** Retrieve the task list; saved tasks are returned.

03 **TC-03:** Edit task details; changes are saved and displayed.

04 **TC-04:** Change completion status; the status is updated.

05 **TC-05:** Move a task to Trash; it is marked deleted and shown there.

06 **TC-06:** Restore a task; it becomes active again.

07 **TC-07:** Permanently delete a task; it is removed.

08 **TC-08:** Export a backup; JSON data is generated.

09 **TC-09:** Import a backup; tasks are restored or imported.

10 **TC-10:** Change settings; preferences are reflected by the application.

If required by an instructor, screenshots or more detailed evidence can be added to document each test result.

Made with GAMMA

08 · EVALUATION

Limitations and practical evaluation

CURRENT LIMITATIONS

01 The project is primarily for local development and demonstration.

02 User authentication and account-based access are not included.

03 Tasks are stored in a local SQLite database.

04 Records are not synchronized across devices.

05 The application depends on the backend server being active.

EVALUATION

Within its intended scope, NexTask demonstrates a complete basic workflow: the user interacts with a browser interface, the frontend calls backend routes, the backend updates persistent records, and the response returns to the interface.

These boundaries should be stated clearly when presenting the application so a local learning project is not mistaken for a hosted multi-user service.

Made with GAMMA

09 · NEXT STEPS

Future scope and conclusion

FUTURE SCOPE

01 Add registration, login, and secure account management.

02 Deploy the application to a cloud hosting platform.

03 Add due-date reminders and notifications.

04 Synchronize tasks across devices.

05 Add search, filtering, and sorting.

06 Support recurring tasks and calendar integration.

07 Improve accessibility and responsive design.

CONCLUSION

NexTask brings task creation, viewing, updating, completion, deletion, restoration, and backup into one interface. It provides practical experience with frontend development, Flask route handling, API communication, SQLite persistence, database design, and Git-based version control.

Its modular foundation supports future authentication, deployment, notifications, accessibility, and synchronization improvements.

Made with GAMMA